## SCHOOL OF TECHNOLOGY

### PROJECT-BASED LEARNING REPORT




# CAMPUS SHUTTLE ROUTING AND CAPACITY SCHEDULING SYSTEM

**DESIGN ANALYSIS AND ALGORITHMS (25TU03MJM1)**

**Bachelor of Technology - Semester III**

**AIML - Rhinos**

**Submitted by**


| Student Name | Roll Number |
|---|---|
| Ishanth M | 25WU0102101 |
| Jack Manuel | 25WU0102102 |


**Under the guidance of**

**Dr. Syed Javed Pasha**
**Assistant Professor, School of Technology**
**Academic Year 2026-27**
**October 2026**

# DECLARATION

We hereby declare that the Project-Based Learning report entitled “Campus Shuttle Routing and Capacity Scheduling System” is an original record of the work carried out by us for the course Design Analysis and Algorithms (25TU03MJM1), under the guidance of Dr. Syed Javed Pasha, Assistant Professor, School of Technology, during the academic year 2026-27.

The system model, algorithm implementations, web application, testing, observations and conclusions presented in this report are based on our project work. Standard algorithm descriptions and technical references used for background study have been acknowledged in the references section. This report has not been submitted, either fully or partly, for any other academic award.

| Student Name | Roll Number | Signature |
|---|---|---|
| Ishanth M | 25WU0102101 | |
| Jack Manuel | 25WU0102102 | |

Place: Hyderabad

Date: October 2026

# CERTIFICATE

This is to certify that the Project-Based Learning report entitled “Campus Shuttle Routing and Capacity Scheduling System” is a bona fide record of work completed by Ishanth M (25WU0102101) and Jack Manuel (25WU0102102), students of Semester III, AIML - Rhinos, School of Technology, for the course Design Analysis and Algorithms (25TU03MJM1) during the academic year 2026-27.

The project was carried out under my supervision and is suitable for submission and evaluation as part of the prescribed Project-Based Learning requirements.

Signature: _____

Dr. Syed Javed Pasha

Assistant Professor, School of Technology

# ACKNOWLEDGEMENT

We express our sincere gratitude to Woxsen University and the School of Technology for providing the academic environment and facilities required to complete this Project-Based Learning activity.

We are deeply thankful to our project guide, Dr. Syed Javed Pasha, Assistant Professor, School of Technology, for his guidance, encouragement and constructive feedback during the planning, algorithm selection, implementation and documentation of the project. His suggestions helped us connect the theoretical concepts of graph algorithms, greedy methods and dynamic programming with a clear transportation problem.

We also thank the faculty members and classmates who reviewed the demonstration and offered useful observations. Finally, we acknowledge the coordinated effort of every team member in dataset preparation, algorithm implementation, interface development, functional testing and report preparation.

Project Team

Ishanth M - 25WU0102101

Jack Manuel - 25WU0102102

# ABSTRACT

Campus shuttle operation is a small but realistic optimization problem because route length, passenger demand, vehicle capacity and trip timing must be considered together. Manual decisions can lead to unnecessary travel, overlapping trips and poor use of available seats. This project presents a Campus Shuttle Routing and Capacity Scheduling System that models seven campus locations as a weighted undirected graph and combines graph, greedy and dynamic-programming techniques in one working localhost application.

Dijkstra's algorithm finds the shortest route between a selected source and destination. Floyd-Warshall computes shortest travel times between every pair of stops, while Kruskal produces a minimum spanning network. Activity Selection chooses compatible shuttle trips. Fractional Knapsack demonstrates partial allocation, and 0/1 Knapsack selects complete passenger groups without exceeding shuttle capacity. Each algorithm addresses a different subproblem and the interface keeps its output easy to trace.

The React and Vite prototype provides separate passenger and administrator dashboards. Passengers can select stops, enter passenger demand, view the highlighted shortest path and start an animated shuttle simulation. The administrator can monitor demand, view fleet status, modify sample inputs and inspect each algorithm result. The supplied campus graph contains seven stops and eleven roads. The demonstrated Main Gate to Student Hostel route takes 13 minutes, the minimum spanning network has total weight 17, and a 40-seat example can select 35 passengers with five seats remaining.

Functional checks compare the displayed results with edge sums, structural conditions and dynamic-programming limits. The project is an academic decision-support prototype that uses fixed sample data. It does not claim real GPS tracking, external map integration or production authentication.

Keywords: campus shuttle, shortest path, Dijkstra, Floyd-Warshall, Kruskal, activity selection, knapsack, route optimization

# TABLE OF CONTENTS


| Contents | Page No. |
|---|---|
| Declaration | 2 |
| Certificate | 3 |
| Acknowledgement | 4 |
| Abstract | 5 |
| Table of Contents | 6 |
| List of Figures | 7 |
| List of Tables | 8 |
| Chapter 1 Introduction | 9 |
| Chapter 2 Problem Statement and Requirements | 11 |
| Chapter 3 Algorithmic Background | 14 |
| Chapter 4 Methodology and System Design | 16 |
| Chapter 5 Algorithm Design and Analysis | 20 |
| Chapter 6 Application Implementation | 23 |
| Chapter 7 Testing and Results | 27 |
| Chapter 8 Conclusion and Future Scope | 29 |
| References | 30 |

# LIST OF FIGURES


| Figure | Title |
|---|---|
| Figure 4.1 | Three-layer system architecture |
| Figure 4.2 | Weighted campus network |
| Figure 4.3 | Integrated processing workflow |
| Figure 6.1 | RouteLab login interface |
| Figure 6.2 | Passenger route-planning dashboard |
| Figure 6.3 | Animated live-map simulation |
| Figure 6.4 | Administrator operations overview |
| Figure 6.5 | Shortest-route and minimum-network result |
| Figure 6.6 | Floyd-Warshall algorithm laboratory |

# LIST OF TABLES


| Table | Title |
|---|---|
| Table 2.1 | Functional requirements |
| Table 2.2 | Non-functional requirements |
| Table 2.3 | Requirement traceability |
| Table 3.1 | Algorithm roles and design paradigms |
| Table 3.2 | Background references and project relevance |
| Table 4.1 | Campus network dataset |
| Table 4.2 | Major system modules |
| Table 4.3 | Input and output model |
| Table 5.1 | Complexity comparison |
| Table 6.1 | Development tools |
| Table 7.1 | Functional test cases |
| Table 7.2 | Verified result summary |

# CHAPTER 1
# INTRODUCTION

## 1.1 Background

Transportation inside a university campus depends on a small network of roads, but the operating decision is not automatically simple. Passengers gather at different stops, demand changes during the day, each shuttle has a fixed capacity and every trip occupies part of a driver shift. A route that looks short on a map may still take longer because travel-time weights differ. A schedule with many trips may also fail if their timings overlap.

The project represents the campus as a weighted undirected graph. Every location becomes a vertex, each road becomes an edge and the expected travel time becomes the edge weight. This model allows established Design and Analysis of Algorithms methods to produce results that can be checked mathematically and demonstrated visually.

## 1.2 Existing Process

A manual shuttle process commonly depends on a fixed route chart and verbal decisions. A passenger may know where the shuttle goes but may not know the quickest connection or whether enough seats remain. Administrators can also find it difficult to compare demand across stops or check whether two planned trips overlap.

- Routes may be selected without comparing total travel time.
- Passenger demand and available seats may be considered separately.
- Overlapping trip timings can create an impossible schedule.
- Repeated route questions require the same manual calculation.
- The reason for a scheduling or allocation decision may not be visible.

## 1.3 Motivation

The project was selected because it connects several syllabus algorithms to one understandable problem. Instead of demonstrating isolated code samples, the system assigns each algorithm a practical responsibility and shows the result through one interface. This makes the implementation easier to evaluate and the theory easier to explain.

The selected scope also supports direct comparison between algorithmic paradigms. A route query demonstrates graph relaxation, trip scheduling demonstrates the greedy-choice property, and capacity allocation demonstrates how dynamic programming evaluates combinations. Using one shared case study makes the differences visible without changing the application domain for every algorithm.

## 1.4 Proposed System

RouteLab is a working local web application with passenger and administrator dashboards. A passenger chooses source and destination stops, enters passenger count and shuttle capacity, and requests a route. The application runs Dijkstra on the stored campus graph, reconstructs the ordered path, highlights it and reports total time. The live-map page moves a shuttle marker along the calculated route as an academic simulation.

The administrator dashboard presents overall demand, shuttle status, route results and an Algorithm Laboratory. Demand and capacity values can be changed so that users can observe how allocation results respond. The application keeps the dataset inside the client application, which removes database and network dependencies during a classroom demonstration.

## 1.5 Scope

- Seven fixed campus stops and eleven weighted road connections.
- Source-to-destination shortest-route calculation.
- All-pairs shortest-distance matrix.
- Minimum spanning network connecting every stop.
- Greedy selection of non-overlapping shuttle trips.
- Fractional and complete-group capacity allocation.
- Separate passenger and administrator dashboards.
- Animated movement along the computed route.

The prototype excludes real GPS, Google Maps, ticket booking, payment, cloud deployment and a production user database. These features are outside the DAA objective and would make the demonstration harder without improving the algorithm analysis.

A deliberate scope boundary is important for correctness. The system reports only what the supplied data can justify: calculated travel time, simulated route progress, selected intervals and capacity use. It does not describe a simulated value as a live sensor reading or treat a minimum spanning tree as a passenger route.

## 1.6 Organisation of the Report

Chapter 2 defines the problem and requirements. Chapter 3 explains the algorithmic background. Chapter 4 presents the methodology, dataset and system design. Chapter 5 describes each algorithm and its complexity. Chapter 6 documents the implementation and user interfaces. Chapter 7 presents testing and verified results. Chapter 8 provides conclusions, recommendations and realistic future extensions.

# CHAPTER 2
# PROBLEM STATEMENT AND REQUIREMENTS

## 2.1 Problem Statement

The problem is to design a campus shuttle decision-support system that reduces avoidable passenger waiting by choosing efficient routes, respecting limited vehicle capacity and selecting trips that can fit inside a shift. The system must use the same weighted graph consistently across its routing and network modules and must present the output in a form that can be explained during a short demonstration.

## 2.2 Project Objectives

1. Model campus stops and roads as a weighted undirected graph.
2. Find the shortest route and travel time between selected stops.
3. Compute shortest travel times between every pair of stops.
4. Construct a connected minimum-weight network without cycles.
5. Select a maximum compatible set of shuttle trips.
6. Allocate limited seats using greedy and dynamic-programming models.
7. Present inputs, intermediate meaning and outputs through a responsive web interface.
8. Verify results using independent conditions rather than accepting screen output alone.

## 2.3 System Users

The passenger role focuses on journey planning. The administrator role focuses on monitoring demand, shuttle state and algorithm results. Both roles use the same fixed campus dataset, but the visible controls reflect their different responsibilities.

Separating the roles also improves the demonstration sequence. A reviewer can first see the practical passenger result and then open the administrator pages to examine how the result was computed, how demand changes affect allocation and how the complete distance matrix supports repeated queries.

## 2.4 Functional Requirements

| ID | Requirement | User |
|---|---|---|
| FR-01 | Authenticate with fixed demonstration credentials. | Both |
| FR-02 | Select valid source and destination stops. | Passenger |
| FR-03 | Calculate and display the shortest route. | Passenger |
| FR-04 | Animate a shuttle along the selected path. | Passenger |
| FR-05 | Display current demand and shuttle status. | Admin |
| FR-06 | Change demand and capacity inputs. | Admin |
| FR-07 | Run and inspect all six algorithms. | Admin |
| FR-08 | Display schedules, matrices and capacity results. | Admin |

## 2.5 Non-Functional Requirements

| Area | Requirement |
|---|---|
| Usability | Controls and outputs must be understandable during a short demonstration. |
| Correctness | The same edge weights must be used by every graph algorithm. |
| Responsiveness | The interface must fit standard laptop and mobile screens. |
| Performance | Results should appear immediately for the seven-stop sample graph. |
| Maintainability | Algorithm functions must remain separate from interface components. |
| Reliability | The application must operate without external APIs or internet data. |

## 2.6 Assumptions and Constraints

Travel times are positive, symmetric and static. The road network is connected. Passenger demand, capacity and trip intervals are known when optimization runs. The animated map shows simulated progress, not measured vehicle coordinates. Fixed credentials support the demonstration only and do not provide production security.

These assumptions keep the mathematical model consistent with the selected algorithms. Positive weights satisfy Dijkstra's requirement, a connected graph permits a complete spanning tree, and known trip intervals allow the standard Activity Selection strategy. If these assumptions change, the algorithm choice must be reviewed instead of applying the current result without modification.

## 2.7 Requirement Traceability

Each major requirement is connected to one implementation module and one visible verification result. This traceability prevents features from appearing only in the written scope without evidence in the application.

| Requirement | Implementation module | Verification evidence |
|---|---|---|
| FR-02, FR-03 | Route Planner | Highlighted path and travel time |
| FR-04 | Live Map | Moving marker and progress value |
| FR-05 | Operations Overview | Demand bars and fleet table |
| FR-06 | Fleet and Demand | Recalculated capacity result |
| FR-07 | Algorithm Laboratory | Six selectable algorithm views |
| FR-08 | Scheduling and matrix views | Selected trips and distance table |

# CHAPTER 3
# ALGORITHMIC BACKGROUND

## 3.1 Weighted Graphs

A weighted graph G=(V,E) contains vertices V and edges E with numerical costs. In this project, vertices represent stops and weights represent travel time in minutes. The cost of a path is the sum of its edge weights. The path containing the fewest edges is not necessarily the path with the lowest time.

## 3.2 Shortest Paths

Dijkstra solves a single-source shortest-path problem when edge weights are non-negative. It is suitable when a passenger asks for one route. Floyd-Warshall solves an all-pairs problem by progressively considering each vertex as an allowed intermediate stop. It is suitable when the system must prepare a complete distance matrix.

## 3.3 Minimum Spanning Tree

A minimum spanning tree connects all vertices with V-1 edges and minimum total weight. Kruskal sorts roads by weight and accepts a road only when it joins different components. The resulting network is a minimum connected backbone. It is not the route every passenger should follow.

This distinction matters because a spanning tree minimizes the combined weight of the selected network, not the travel time of every possible journey. Removing non-tree roads may increase individual route lengths even while the overall network cost becomes smaller.

## 3.4 Greedy Algorithms

A greedy method commits to a locally preferred choice. Activity Selection chooses the compatible trip that finishes first, preserving as much time as possible for later trips. Fractional Knapsack ranks groups by priority per seat and permits partial selection. Greedy reasoning is used only where the problem assumptions support it.

## 3.5 Dynamic Programming

Dynamic programming stores solutions to overlapping subproblems. Floyd-Warshall reuses distances computed with earlier intermediate vertices. The 0/1 Knapsack table stores the best priority value for each prefix of passenger groups and each capacity. This approach can reject an attractive local choice when another combination has a better total value.

Both dynamic-programming modules follow the same general idea but use different state definitions. Floyd-Warshall indexes states by source, destination and permitted intermediate stops. Knapsack indexes states by the number of groups considered and the remaining seat limit. The state must match the decision being optimized.

## 3.6 Algorithm Roles

| Algorithm | Paradigm | Project role |
|---|---|---|
| Dijkstra | Graph | One requested shortest route |
| Floyd-Warshall | Dynamic programming | Shortest distance for every stop pair |
| Kruskal | Greedy graph | Minimum connected network |
| Activity Selection | Greedy | Compatible shuttle trips |
| Fractional Knapsack | Greedy | Partial allocation by priority density |
| 0/1 Knapsack | Dynamic programming | Complete groups within capacity |

## 3.7 Background Sources

| Source | Project relevance |
|---|---|
| Cormen et al. | Shortest paths, MST, greedy and DP foundations. |
| Dijkstra | Original shortest-path method for non-negative networks. |
| Floyd and Warshall | All-pairs recurrence using intermediate vertices. |
| Kruskal | Minimum spanning tree by increasing edge cost. |
| React documentation | Component state and event-driven interface. |
| Vite documentation | Local development and production build workflow. |

# CHAPTER 4
# METHODOLOGY AND SYSTEM DESIGN

## 4.1 Development Approach

An iterative prototype approach was used. The team first fixed the campus dataset and verified expected graph results manually. Pure JavaScript functions were then written for every algorithm. After the outputs agreed with the sample calculations, passenger and administrator interfaces were connected to the functions. Final work focused on responsive layout, result clarity and repeatable demonstration inputs.

9. Define project scope and fixed campus dataset.
10. Implement and test each algorithm independently.
11. Build passenger journey-planning workflow.
12. Build administrator monitoring and algorithm views.
13. Connect editable demand and capacity inputs.
14. Compile the production build and prepare functional checks.

## 4.2 System Architecture

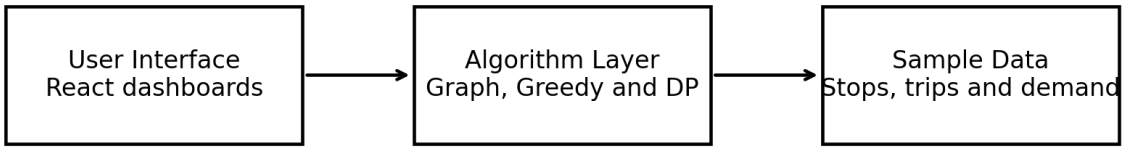



*Figure 4.1 Three-layer system architecture*

The interface layer manages input and visual output. The algorithm layer performs all optimization calculations. The sample-data layer stores stops, edges, trips, demand and fleet information. This separation prevents presentation code from replacing or hiding the algorithm logic.

When a control changes, React updates the relevant state and recalculates only the derived result needed by the visible page. The graph itself remains the common source of truth. This avoids the serious error of displaying one set of travel times while an algorithm silently uses another.

## 4.3 Campus Network Dataset

| Edge | Connection | Time min |
|---|---|---|
| A-G | Main Gate - Admin Block | 3 |
| A-B | Main Gate - Academic Block | 5 |
| G-B | Admin Block - Academic Block | 2 |
| G-C | Admin Block - Library | 4 |
| B-C | Academic Block - Library | 3 |
| B-D | Academic Block - Cafeteria | 6 |
| C-D | Library - Cafeteria | 2 |
| C-E | Library - Student Hostel | 7 |
| D-E | Cafeteria - Student Hostel | 4 |
| D-F | Cafeteria - Sports Complex | 5 |
| E-F | Student Hostel - Sports Complex | 3 |

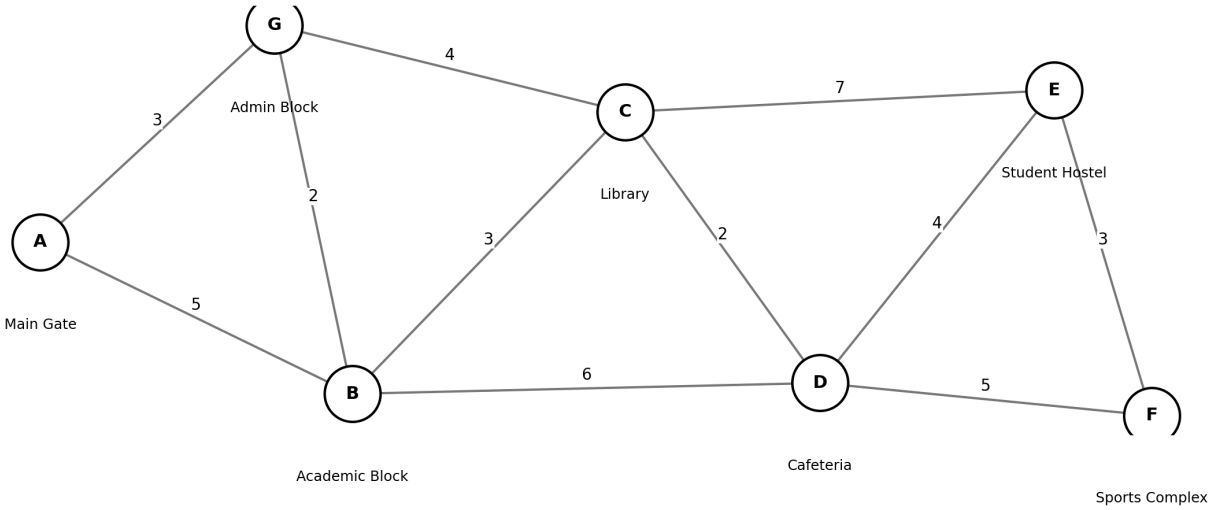



*Figure 4.2 Weighted campus network*

## 4.4 Major System Modules

| Module | Responsibility |
|---|---|
| Authentication | Select passenger or administrator and verify demo credentials. |
| Route Planner | Accept route inputs and display Dijkstra result. |
| Live Map | Animate a shuttle marker on the calculated path. |
| Operations Overview | Summarise demand, fleet state and selected trips. |
| Network Map | Compare shortest-route and minimum-network outputs. |
| Fleet and Demand | Edit demand and capacity and show allocation. |
| Algorithm Laboratory | Expose every algorithm result separately. |
| Scheduling | Display selected and overlapping trip intervals. |

## 4.5 Use-Case Summary

| Actor | Use case | Expected result |
|---|---|---|
| Passenger | Log in | Passenger dashboard opens. |
| Passenger | Plan journey | Shortest route and time appear. |
| Passenger | Start simulation | Marker moves along selected path. |
| Admin | View overview | Metrics, demand and fleet appear. |
| Admin | Adjust capacity | Knapsack results recalculate. |
| Admin | Inspect algorithm | Matrix, tree, schedule or allocation appears. |

## 4.6 Input and Output Model

| Input | Rule | Output affected |
|---|---|---|
| Source stop | Valid stop identifier | Ordered shortest path |
| Destination stop | Different valid identifier | Total travel time |
| Passenger count | Positive integer | Boarding and waiting count |
| Shuttle capacity | Positive integer | Used and remaining seats |
| Demand by stop | Non-negative integers | Fractional and 0/1 selections |
| Trip intervals | Start before finish | Compatible selected trips |

## 4.7 Processing Workflow

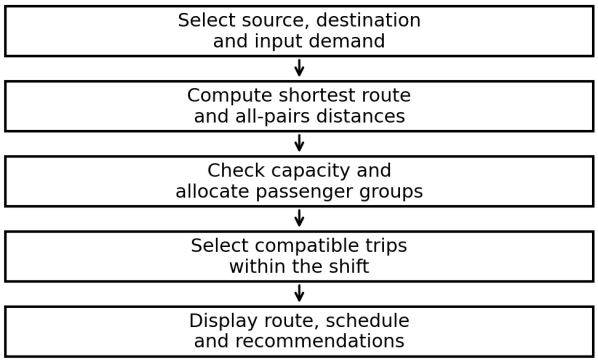



*Figure 4.3 Integrated processing workflow*

## 4.8 Validation Rules

- Source and destination must represent different known stops.
- Edge weights and capacities must remain positive.
- Passenger demand must not be negative.
- A trip start time must occur before its finish time.
- 0/1 Knapsack cannot split a passenger group.
- Activity Selection must reject intervals that overlap the last selected trip.

Validation messages are kept close to the relevant control. Invalid input is rejected before algorithm execution, so the interface does not present an impossible value as an optimization result. This separation also makes test failures easier to locate.

# CHAPTER 5
# ALGORITHM DESIGN AND ANALYSIS

## 5.1 Dijkstra Algorithm

The algorithm initializes the selected source with distance zero and every other stop with infinity. It repeatedly chooses the unvisited stop with minimum tentative distance and relaxes each neighboring road. A predecessor map records how each best distance was reached. After the destination is finalized, the predecessor chain is reversed to reconstruct the path.

For Main Gate A to Student Hostel E, the result is A-G-C-D-E. The time equals 3+4+2+4=13 minutes. The alternative A-B-C-D-E takes 14 minutes, so it is not selected.

**Dijkstra pseudocode**

```
dist[source] = 0; set every other distance to infinity
while an unvisited vertex remains:
   choose u with minimum dist[u]
   for each neighbour v of u:
      if dist[u] + weight(u,v) < dist[v]:
         update dist[v] and predecessor[v]
reconstruct destination path using predecessors
```

## 5.2 Floyd-Warshall Algorithm

The initial matrix stores zero on the diagonal, direct road weights for connected stops and infinity otherwise. For every possible intermediate stop k, each pair i,j is updated using D[i,j]=min(D[i,j], D[i,k]+D[k,j]). The final matrix answers repeated distance queries and independently confirms the Dijkstra distance for the same pair.

For seven stops, the matrix contains forty-nine values and the cubic computation is small. The method becomes more expensive as the stop count grows, but it remains useful when many route questions are expected because every pair can be answered from the completed table.

## 5.3 Kruskal Algorithm

Kruskal sorts all roads from smallest to largest. A Disjoint Set Union structure tracks connected components. An edge is accepted only when its endpoints belong to different components. Six accepted edges connect the seven stops without a cycle. Their weights 2, 2, 3, 3, 3 and 4 produce a total weight of 17.

**Kruskal pseudocode**

```
sort all edges by increasing weight
make a separate set for every vertex
for each edge (u,v,w):
   if find(u) differs from find(v):
      accept the edge
      union the two components
stop after V-1 accepted edges
```

## 5.4 Activity Selection

Candidate trips are sorted by finishing time. The first finishing trip is selected. Every later trip is accepted only if its start time is greater than or equal to the finish time of the last accepted trip. The sample schedule chooses three non-overlapping trips. The approach maximizes the number of compatible activities under the standard model.

The model assumes one shuttle or one shared resource for the selected schedule and gives every trip equal importance. If trips have different profits, passenger counts or service priorities, weighted interval scheduling would be more appropriate than the standard greedy rule.

## 5.5 Correctness Conditions

- Dijkstra requires non-negative edge weights.
- Kruskal must accept exactly V-1 edges for a connected graph.
- The spanning result must contain no cycle.
- Every selected activity must start after the prior activity finishes.
- A shortest-path total must equal the sum of its selected edge weights.

## 5.6 Fractional Knapsack

Each passenger group has a seat requirement and priority. The fractional method computes priority per seat, sorts groups by this ratio and fills capacity from the highest ratio downward. If the next group is larger than the remaining capacity, only the required part is selected. This model is useful as an academic comparison, but splitting an organised passenger group may not be operationally appropriate.

## 5.7 0/1 Knapsack

The 0/1 method either accepts or rejects each complete group. DP[i,c] stores the maximum priority achievable from the first i groups within capacity c. If group i is too large, the earlier value is retained. Otherwise, the algorithm compares rejection with selection plus the best value for the remaining capacity. Backtracking identifies the chosen groups.

## 5.8 Complexity Analysis

| Algorithm | Time | Space | Use |
|---|---|---|---|
| Dijkstra | $O(V^2)$ in prototype | O(V) | Single route query |
| Floyd-Warshall | $O(V^3)$ | $O(V^2)$ | All-pairs preprocessing |
| Kruskal | O(E log E) | O(V) | Minimum network |
| Activity Selection | O(n log n) | O(n) | Trip scheduling |
| Fractional Knapsack | O(n log n) | O(n) | Partial allocation |
| 0/1 Knapsack | O(nC) | O(nC) | Whole-group allocation |

The sample network is small, so measured runtime is not meaningful as a performance claim. Complexity remains important because it explains how each module would scale when stops, edges, groups or capacity increase.

# CHAPTER 6
# APPLICATION IMPLEMENTATION

## 6.1 Development Tools

| Tool | Use |
|---|---|
| React | Builds reusable interface components and state-driven views. |
| JavaScript | Implements the six algorithms and interaction logic. |
| Vite | Runs the local development server and production build. |
| HTML and CSS | Provide semantic structure and responsive visual styling. |
| SVG | Draws roads, nodes, weights and shuttle marker. |
| Local sample data | Removes database and external-service dependencies. |

## 6.2 Module Organisation

The application keeps fixed stops, weighted roads, trips and fleet information as JavaScript objects. Dedicated functions construct adjacency structures, calculate routes, build the all-pairs matrix, select MST edges, choose compatible trips and solve capacity allocations. React components call these functions and render only the resulting state.

The routing function returns both the distance and ordered path instead of returning only a number. The map uses the ordered path to identify active edges and interpolate the shuttle marker position. Allocation functions return chosen groups, used seats and remaining seats so the dashboard can explain the result without repeating the algorithm inside a component.

## 6.3 Demonstration Authentication

The shared login screen allows the evaluator to choose Passenger or Administrator. Fixed credentials open the corresponding dashboard. This is intentionally a classroom demonstration mechanism and must not be described as secure production authentication because no server or user database validates the account.

## 6.4 Responsive Interface

The desktop interface uses a fixed sidebar, page header, cards and a large map area. At smaller widths, navigation moves to the bottom and multi-column sections stack vertically. The visual design uses dark green, white and light neutral backgrounds to keep route highlights and data values readable.

## 6.5 Login and Passenger Route Planning

Figure 6.1 shows the shared entry screen. The left side introduces the project and displays the sample scope. The right side contains role selection, credentials and a clear button for opening the selected dashboard.

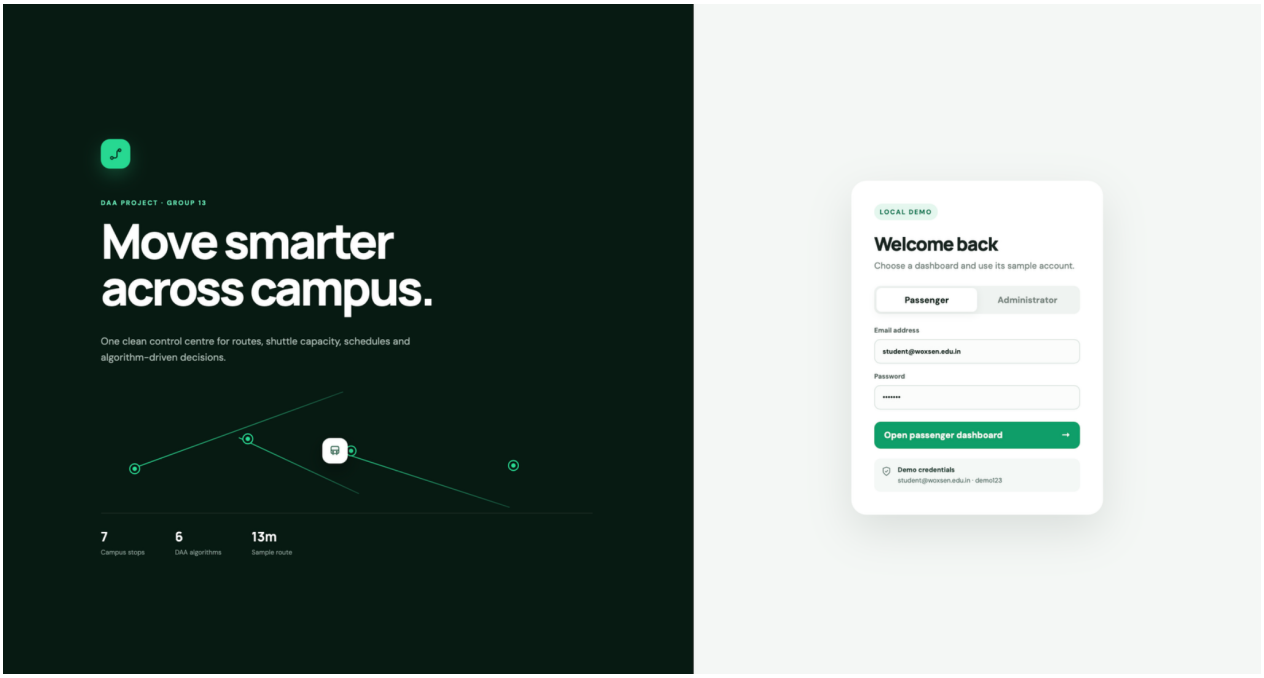



*Figure 6.1 RouteLab login interface*

After passenger login, the route planner accepts source, destination, passenger count and capacity. Figure 6.2 shows Main Gate to Student Hostel. The green route is the sequence selected by Dijkstra and the result panel reports 13 minutes.

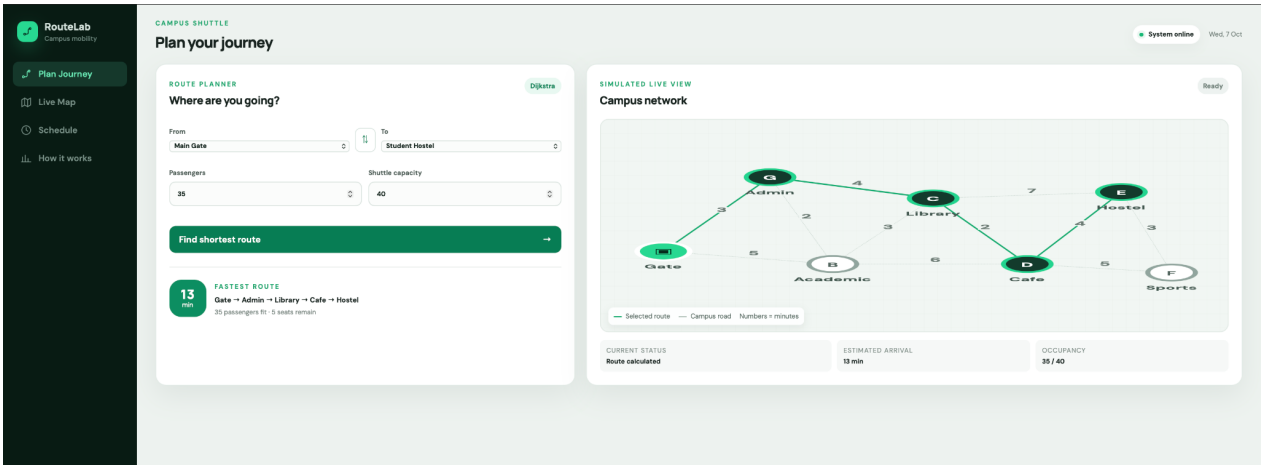



*Figure 6.2 Passenger route-planning dashboard*

## 6.6 Live Map Simulation

The live-map view animates the shuttle marker along the already calculated path. Progress, estimated arrival and occupancy update during the animation. The movement is generated from route geometry and elapsed animation progress; it does not represent a physical GPS reading.

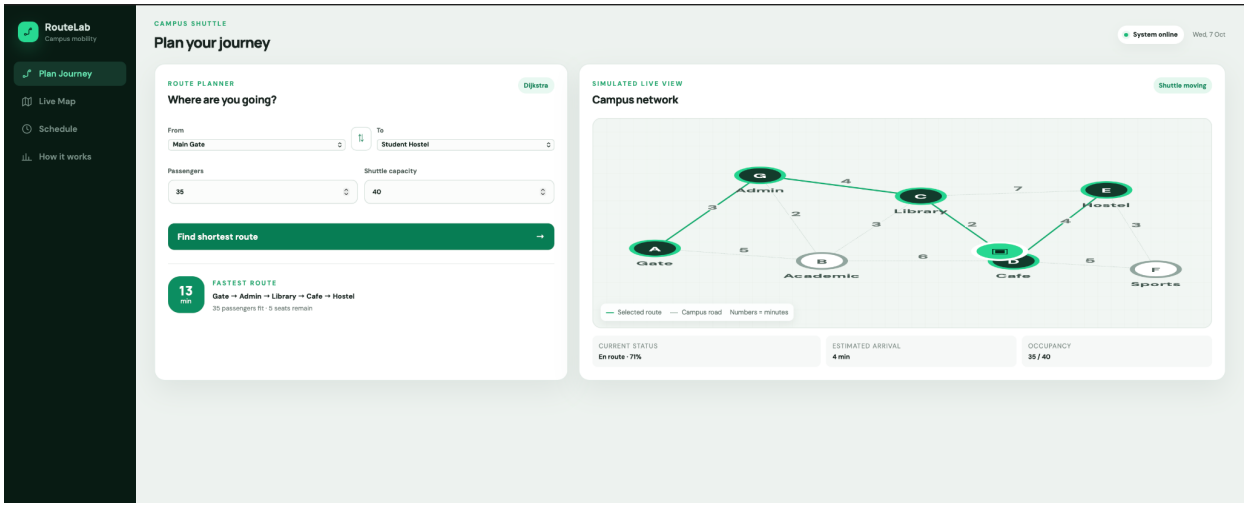



*Figure 6.3 Animated live-map simulation*

## 6.7 Administrator Overview

The administrator overview combines total waiting passengers, active shuttles, average route time, selected trips, demand bars and fleet status. These values come from the fixed demonstration data and editable in-memory inputs.

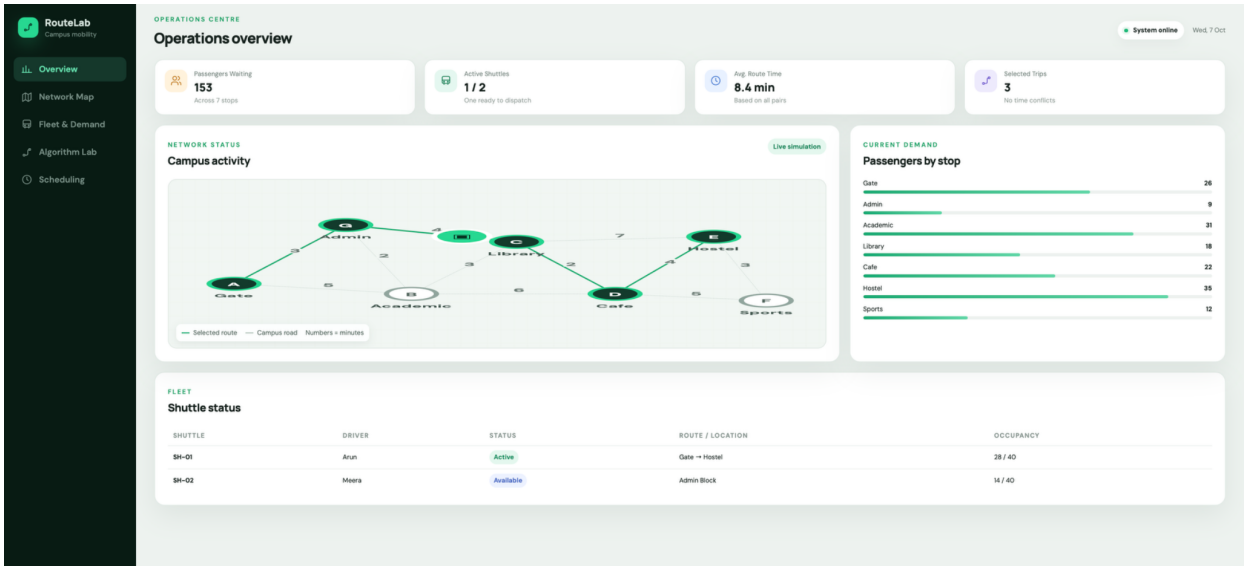



*Figure 6.4 Administrator operations overview*

## 6.8 Network and Algorithm Laboratory

The Network Map page combines two related but different graph outputs. Green edges indicate the requested Dijkstra path. The lower result strip reports both shortest-route time and Kruskal minimum-network cost, preventing the two objectives from being confused.

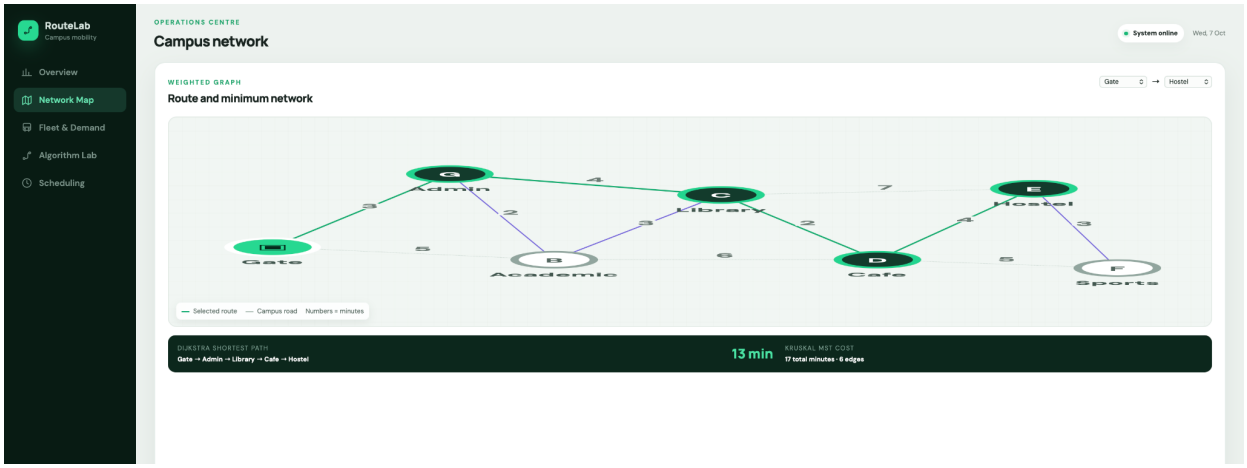



*Figure 6.5 Shortest-route and minimum-network result*

The Algorithm Laboratory exposes one method at a time. Figure 6.6 shows the Floyd-Warshall matrix. Rows and columns correspond to stop codes, and each cell stores the shortest travel time between the selected pair.

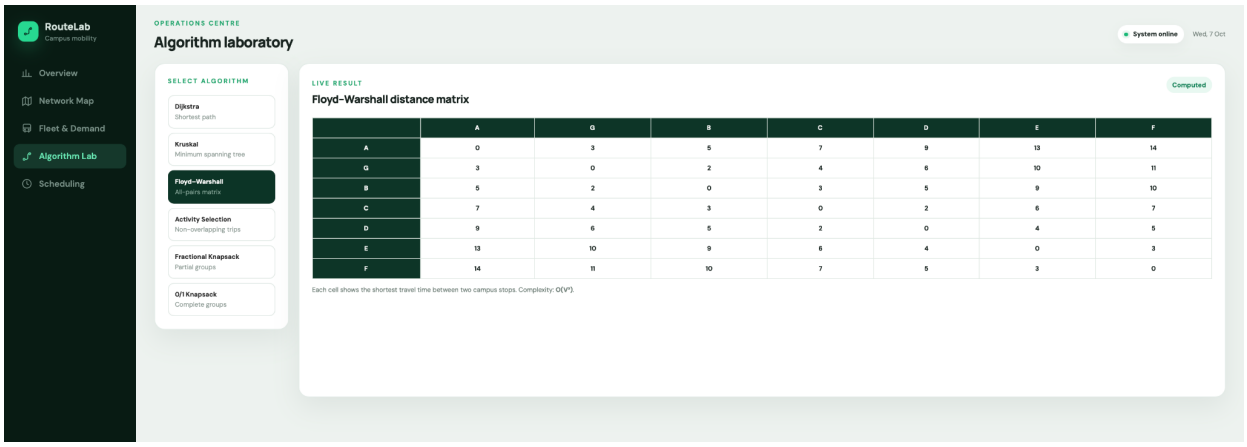



*Figure 6.6 Floyd-Warshall algorithm laboratory*

# CHAPTER 7
# TESTING AND RESULTS

## 7.1 Testing Approach

Testing focuses on algorithm outputs and user-visible workflows. Each result is checked with an independent condition: route weights are summed, the MST is checked for edge count and cycles, selected trips are checked for overlap, and capacity results are checked against the stated limit.

The tests use fixed inputs so that the expected output can be calculated before running the application. This is more reliable than judging correctness only from whether a screen loads. Interface checks confirm navigation and animation, while algorithm checks confirm the values behind the presentation.

| ID | Test | Expected | Status |
|---|---|---|---|
| TC-01 | Passenger demo login | Passenger dashboard opens | Pass |
| TC-02 | Admin demo login | Operations overview opens | Pass |
| TC-03 | A to E route | A-G-C-D-E, 13 min | Pass |
| TC-04 | Same source and destination | Request prevented | Pass |
| TC-05 | Kruskal full graph | 6 edges, total 17 | Pass |
| TC-06 | Floyd matrix A to E | Distance 13 | Pass |
| TC-07 | Sample trip set | 3 compatible trips | Pass |
| TC-08 | Capacity 40 | 35 selected, 5 free | Pass |
| TC-09 | Start live map | Marker advances on route | Pass |

## 7.2 Verified Result Summary

| Result | Output | Verification |
|---|---|---|
| Shortest route | A-G-C-D-E; 13 min | Selected edge sum equals 13 |
| Minimum network | 6 edges; weight 17 | Connected and acyclic |
| All-pairs result | A to E equals 13 | Matches Dijkstra result |
| Trip schedule | 3 selected trips | No selected intervals overlap |
| Capacity | 35 used; 5 free | Does not exceed 40 seats |

# CHAPTER 8
# CONCLUSION AND FUTURE SCOPE

## 8.1 Conclusion

The Campus Shuttle Routing and Capacity Scheduling System demonstrates how several DAA techniques can solve separate parts of one transportation problem. Dijkstra and Floyd-Warshall calculate travel decisions, Kruskal identifies a minimum connected network, Activity Selection removes trip conflicts, and the two knapsack models demonstrate alternative capacity rules. The working React application makes each result visible and traceable.

The final results also show why algorithm selection must follow the exact question. Dijkstra cannot replace a scheduler, Kruskal cannot answer an individual route query, and a greedy capacity rule cannot guarantee the same whole-group result as dynamic programming. Combining the modules is useful only because their responsibilities remain separate.

The prototype meets its academic objective because it is small enough to explain but complete enough to demonstrate input, algorithm execution and output. The use of fixed sample data keeps the results reproducible and removes failure points unrelated to DAA.

## 8.2 Policy Recommendations

- Increase shuttle frequency when hostel or academic-stop demand is high.
- Assign larger capacity to stops with consistently higher demand.
- Prioritize passengers who have waited longer when priorities are available.
- Avoid assigning one shuttle to overlapping trip intervals.
- Recalculate routes whenever the travel-time dataset changes.

## 8.3 Limitations

- Travel times do not change with traffic or weather.
- The live marker is simulated rather than GPS-based.
- Credentials and data are stored for demonstration, not production use.
- The current model does not handle multiple vehicles simultaneously.
- Passenger groups use simplified priority values.

## 8.4 Future Scope

Future work may add a small persistent database, authenticated accounts, live vehicle coordinates, time-dependent edge weights and multi-shuttle assignment. These extensions should be added only after preserving the current separation between routing, scheduling, allocation and presentation logic.

# REFERENCES


[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed. MIT Press, 2022.

[2] E. W. Dijkstra, “A note on two problems in connexion with graphs,” Numerische Mathematik, vol. 1, pp. 269-271, 1959.

[3] R. W. Floyd, “Algorithm 97: Shortest path,” Communications of the ACM, vol. 5, no. 6, p. 345, 1962.

[4] S. Warshall, “A theorem on Boolean matrices,” Journal of the ACM, vol. 9, no. 1, pp. 11-12, 1962.

[5] J. B. Kruskal, “On the shortest spanning subtree of a graph and the traveling salesman problem,” Proceedings of the American Mathematical Society, vol. 7, no. 1, pp. 48-50, 1956.

[6] R. Bellman, Dynamic Programming. Princeton University Press, 1957.

[7] React Documentation, “Describing the UI and managing state.” Available: https://react.dev/. Accessed: October 2026.

[8] Vite Documentation, “Getting Started.” Available: https://vite.dev/guide/. Accessed: October 2026.

[9] MDN Web Docs, “SVG: Scalable Vector Graphics.” Available: https://developer.mozilla.org/docs/Web/SVG. Accessed: October 2026.

[10] NetworkX Documentation, “Algorithms.” Available: https://networkx.org/documentation/stable/reference/algorithms/. Accessed: October 2026.

[11] A. V. Aho, J. E. Hopcroft and J. D. Ullman, Data Structures and Algorithms. Addison-Wesley, 1983.

[12] S. Dasgupta, C. Papadimitriou and U. Vazirani, Algorithms. McGraw-Hill, 2006.